

RIPHAH INTERNATIONAL UNIVERSITY



Software Engineering

Report

Project Title: Campus Hub

Batch: Fall 2023

Project Team Members

Name of Students	Sap ID	Program	Email Address
Hoor Spogmay	53541	BS-SE	53541@students.riphah.edu.pk
Aqsa Ahmed	53106	BS-SE	53106@students.riphah.edu.pk
Simaab Malik	54910	BS-SE	54910@students.riphah.edu.pk

Submitted to

Ma'am Sabahat Ajaz

Date of Submission

09/07/2024

Table of Contents

Project Proposal	6
Introduction:	6
Purpose of the Project:	6
Features of the Project:	7
For Admin:	7
For Teachers:	7
For Students:	7
Technologies Used:	8
Division of Responsibilities:	8
Signed Consent:	9
Conclusion:	10
Process Model:	11
Principles of Agile:	11
Flexibility:	11
Customer Collaboration:	11
Quick Delivery:	11
Resource Efficiency:	11
Process Flow in Agile:	12
Initial Planning :	12
Iterations/Sprints :	12
Feedback Loop :	12
Testing :	12
Agile Method:	12
Scrum will be used because :	12

Benefits of using scrum :	13
The Process Flow:	13
Scrum Roles:	14
Division of Responsibilities:	14
Conclusion:	15
SRS Document	16
Introduction	16
Purpose	16
Scope	16
Definitions, Acronyms and abbreviations	16
References	16
Overall Description	17
Product Perspective	17
Product Features	17
User Classes and Characteristics	17
Operating Environment	17
Design and Implementation Constraints	17
System Features	18
Admin Capabilities	18
Teacher Capabilities	18
User Capabilities	18
. External Interface Requirements	19
User Interfaces	19
Non-Functional Requirements	19
Performance	19
Security	19

Usability.....	19
Reliability.....	19
Scalability.....	20
Use Cases	20
Student Record Management.....	20
Course Allocation Management.....	20
Financial Management	21
Attendance Management	21
Announcement Management.....	21
Backlog.....	22
Product Backlog.....	22
Sprint Backlog.....	23
Glossary.....	24
References	24
Diagrams.....	25
Context Diagram:	25
Sequence Diagram:	27
Architecture:	31
Division of Responsibilities:	31
Implementation Phase :.....	32
Create Student Record CRUD Functionality	32
Display Student Data :	32
2. Student Information :	41
3. Edit Student Information:.....	63
Design Phase :.....	87
Login Page :	88

Student Home Page :	88
Teacher Home Page :	88
CRUD :	89
Division of Responsibilities:	89
Test Cases	90
Conclusion:	95

Project Proposal

Introduction:

This software, called Campus Hub is an Education Management System (EMS), it will be created to make maintaining student data easier and more efficient administratively. It will offer educational institutions an all-in-one platform to effectively manage fee payments, track attendance, and handle student records.

Purpose of the Project:

The purpose of this project is to overcome problems faced by educational systems and provide following facilities.

- 1. Centralized Student Data Management:** To offer a safe and well-organized platform for the management and archiving of academic records, personal data, and student records.
- 2. Streamlined Attendance Tracking:** To make it simple to record, track, and report student attendance while maintaining accuracy and reducing human error.
- 3. Automated Fee Management:** To increase the efficiency of financial administration by streamlining the management of fee structures, payment monitoring, and outstanding balances.
- 4. Improved Accessibility:** To guarantee that all required student data is easily available to authorized personnel, including teachers, administrators, and staff, thereby enhancing communication and decision-making.
- 5. Instructor Dashboard:** For handling both personal and work-related data. Post reminders and essential updates for students. Distribute the deadlines and assignments. Manage the details and courses that have been assigned to you. Exchange study guides and lesson ideas.

Features of the Project:

For Admin:

1.Course Allocation: The teacher can allocate courses for students that will be shown to them.

2.Fee Management: Manages the financial transactions related to student fees, including payment records and overdue fees.

3.Timetable Management: Administrators can create and manage class timetables for both students and teachers.

4. Student Record Management: Administrators will be able to manage all the data entered by students their records etc.

For Teachers:

1.Announcemnt: Can make announcement that will be shown to the students.

2.Course Management: Teachers can manage their assigned courses, including adding materials, setting assignments, and grading.

3.Viewing Student Data: Teachers can access and view students' data, such as academic records, attendance, and personal information.

For Students:

1. Student Record Management: Students will be able to keep track of his/ her official record.

2. Attendance Tracking: Facilitates the easy recording and viewing of student attendance for educators and administrators.

3. Fee Management: Oversees and records tuition payments, schedules of fees, and unpaid amounts.

4. Personal Information Management: Guarantees the safe and convenient storage of student contact information, guardian information, and other personal data.

5. Student Courses: Students can view available courses and register for them.

Technologies Used:

The technology used in this project is Java.

Division of Responsibilities:

- ❖ **Aqsa Ahmed** was responsible for refining the content and preparing the written to effectively communicate the project's features. Also provided an overview ensuring a clear and cohesive understanding of its scope and purpose.
- ❖ **Simaab Malik** provided the necessary content and details needed for the project. Ensured that all relevant information was well organized and aligned with the project's requirements.
- ❖ **Hoor Spogmay** took charge of generating the idea , conducting research and finalizing the document. Played a key role in analyzing the project's feasibility and ensuring its alignment with current educational needs.

Signed Consent:

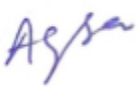
- Hoor Spogmay



-
- Simaab Malik



- Aqsa Ahmed



Conclusion:

The development of this Campus Hub will lead to a significant improvement in the efficiency and accuracy of managing student data and school administrative tasks. By centralizing all student information, institutions can streamline their operations, reducing manual work and minimizing errors. The automated features, like attendance tracking and fee management, will save time for both teachers and administrators, allowing them to focus more on core educational activities. Enhanced accessibility ensures that authorized personnel can easily retrieve relevant information, leading to better decision-making and smoother communication across departments. Ultimately, this will foster a more organized, secure, and user-friendly environment for managing education-related processes.

Process Model:

Developing any software at a small scale “ **Agile** ” methodology is a suitable choice.

Principles of Agile:

- ✓ **Customer Involvement**
- ✓ **Incremental Delivery**
- ✓ **Embrace Change**
- ✓ **Maintain Simplicity**

Using Agile for small-scale development is efficient because Agile process Model provides:

Flexibility:

Agile provides flexibility because developers can make the software efficiently in small sprints called “iterations”. This helps in making a working product faster and according to user feedback.

Customer Collaboration:

As discussed earlier it requires more user involvement, which can be helpful in refining the product timelier and more efficiently.

Quick Delivery:

Instead of delivering the whole product at once. Agile offers delivery of core modules earlier and then adding further advanced features.

Resource Efficiency:

For small teams or projects with limited resources , Agile help us to prioritize features based on importance , ensuring the most valuable parts of the LMS are delivered first.

Process Flow in Agile:

Initial Planning :

Define the basic requirements (admin responsibilities, teacher responsibilities, student responsibilities etc.).

Iterations/Sprints :

Develop the software in small cycles. Each sprint may focus on delivering a specific part of the software (e.g., login system in one sprint, course creation in another).

Feedback Loop :

After each sprint, get feedback from the users (teachers, administrators) to refine the next iteration.

Testing :

Continuous testing is integrated within each sprint to ensure each part is functional before moving to the next.

Agile Method:

Scrum is an agile method that focuses on managing iterative development rather than specific agile practices.

Scrum will be used because :

1. It requires a smaller team (<10 people)
2. Duration of one sprint is small
3. The increments are visible and usable
4. It is time boxed

Benefits of using scrum :

- Increase employee productivity and happiness
- Improve product quality
- Reduce time to market
- Enhance stakeholder satisfaction

The Process Flow:

1. Start with a Product Backlog :

The Product Owner will create and prioritize the backlog.

2. Sprint Planning :

The team will select items from the backlog for the upcoming sprint.

3. Execution :

The team will work on the selected items, meeting daily for stand-ups.

4. Review and Feedback :

At the end of the sprint, the team will share their progress with each other and discuss any problems faced

5. Reflect and Improve :

The team will discuss what worked and what didn't during the sprint retrospective.

6. Repeat :

The process will continue with the next sprint, incorporating feedback and improvements.

Scrum Roles:

1. Product Owner

Hoor Spogmay

2. Scrum Master

Aqsa Ahmed

3. Scrum Team

- Aqsa Ahmed
- Hoor Spogmay
- Simaab Malik

Division of Responsibilities:

❖ Aqsa Ahmed :

Responsible for data collection and analysis.

❖ Simaab Malik :

In charge of content management and resource coordination.

❖ Hoor Spogmay :

Oversees the finalization of the the artifact.

Conclusion:

Scrum is a powerful framework that helps teams navigate the complexities of project management, particularly in dynamic environments like software development. By emphasizing collaboration, adaptability, and continuous improvement, Scrum enables teams to deliver high-quality products that meet user needs effectively.

SRS Document

Introduction

Purpose

The purpose of this document is to describe the software requirements for the software **campus hub**. It is an Education management System (EMS). This system will provide an all-in-one platform for educational institutions to manage student records, track attendance, handle fee payments, and store data efficiently.

Scope

Campus hub shall streamline administrative processes, reduce human errors, and improve decision making by centralizing student data. It will support administrators, teachers and students in managing data related to courses, fees, attendance, and personal information. This app will be developed using Java technology.

Definitions, Acronyms and abbreviations

- **EMS:** Education Management System
- **CRUD:** Create, Read, Update, Delete
- **UI:** User Interface
- **SRS:** Software Requirement Specifications

References

This SRS document is based on the project proposal submitted (**Campus Hub**)

Overall Description

Product Perspective

Campus Hub shall serve as a centralized system that integrates with an institution's existing infrastructure. It will offer features to manage student data, course information, attendance tracking, and fee management. It provides roles-based access to administrators, students and teachers

Product Features

- **Centralized Student Data Management:** Securely store and manage academic and personal information.
- **Attendance Tracking:** Record and report student attendance.
- **Fee Management:** Manage and track student fee payments.
- **Course Management:** Allocate and manage courses for students and teachers.
- **Instructor Dashboard:** Manage assignments, make announcements, and monitor student progress.

User Classes and Characteristics

- **Administrators**
- **Teachers**
- **Students**

Operating Environment

Campus Hub will be a desktop based app that will be accessible on any type of desktop computers or laptops.

Design and Implementation Constraints

- The system will be developed using **Java** technology.
- The system must comply with educational data protection regulations and ensure role-based access.

System Features

Admin Capabilities

- 1. Course Assignment:** Admins can assign courses to educators and students, including adjusting course details as necessary
- 2. Financial Oversight:** Admins shall be able to manage all financial transactions related to student fees, including tracking payments and overdue balances with automated notifications.
- 3. Schedule Coordination:** Admins shall be able to create, modify, and view class schedules for faculty and students, ensuring optimal resource allocation.
- 4. Record Maintenance:** Admins shall be able to update and maintain student academic and personal records, including attendance data, with a secure audit trail.

Teacher Capabilities

- 1. Notification System:** Teachers shall be able to post announcements visible to their students and set reminders for important deadlines.
- 2. Course Oversight:** Teachers shall be able to manage their courses by uploading materials, setting assignments, and grading submissions, with the ability to track student progress.
- 3. Access to Student Information:** Teachers shall be able to review individual student records, including academic performance, attendance, and engagement metrics.

User Capabilities

- 1. Personal Record Management:** Students shall be able to manage and update their academic and personal information.
- 2. Attendance Review:** Students shall be able to access and track their attendance records and receive alerts for any discrepancies.
- 3. Fee Monitoring:** Students shall be able to check their payment history, outstanding fees, and receive notifications for upcoming payments.
- 4. Contact Information Management:** Students shall be able to update their personal and guardian contact details.

5. Course Registration: Students shall be able to enroll in courses, view related materials, and manage their schedules.

. External Interface Requirements

User Interfaces

- A clean, intuitive user interface designed for non-technical users.
- Different dashboards for admins, teachers, and students based on role-based access
- Interactive views for student records, attendance, and fee management.

Non-Functional Requirements

Performance

- The system shall be able to support at least 50 concurrent users without significant performance degradation.
- The system response time should not exceed 2 seconds for most operations

Security

- All sensitive data (student records, fee information) will be encrypted.
- Role-based access control must be implemented to ensure data confidentiality.

Usability

- The system must be easy to use with minimal training, especially for non-technical users like students and teachers.

Reliability

- The system must have 99.9% uptime.
- Automated backups must occur daily to prevent data loss.

Scalability

- The system must be scalable to accommodate more users and data as the institution grows.

Use Cases

Student Record Management

- **Actors:** Admin, Teachers, Students
- **Precondition:** The user must be logged in with the correct permissions.
- **Main Flow:**
 - The Admin logs into the system and goes to the "Student Records" section.
 - The Admin can add, update, or remove student information (personal details, academic history).
 - Teachers can access records of students in their classes.
 - Students can view their personal records.
- **Postcondition:** The student information is updated in the database.

Course Allocation Management

- **Actors:** Admin, Teachers, Students
- **Precondition:** The user must be logged in with the appropriate role.
- **Main Flow:**
 - The Admin logs in and navigates to the "Course Allocation" section.
 - The Admin assigns courses to students and teachers.
 - Teachers manage the courses they are assigned to.

- Students can browse available courses and enroll.
- **Postcondition:** Course assignments are successfully recorded and visible to the respective users.

Financial Management

- **Actors:** Admin, Students
- **Precondition:** The user must be authenticated with the proper permissions.
- **Main Flow:**
 - The admin logs in and accesses the "Fee Management" section.
 - The admin updates the fee structure, logs payments, and monitors unpaid fees.
 - Students can check their payment history and see any outstanding fees.
- **Postcondition:** The financial status is accurately updated in the system.

Attendance Management

- **Actors:** Admin, Teachers, Students
- **Precondition:** The user must be authenticated with the correct role.
- **Main Flow:**
 - Attendance is recorded in the system by either Teachers or the Admin.
 - Students can view their individual attendance records.
 - Admins can generate comprehensive attendance reports.
- **Postcondition:** Attendance data is saved and accessible to relevant parties.

Announcement Management

- **Actors:** Teachers, Students

- **Precondition:** The user must be logged in with the necessary role.
- **Main Flow:**
 - A Teacher creates an announcement and submits it.
 - The announcement is displayed for all students assigned to that class.
 - Students can view the announcement in their dashboards.
- **Postcondition:** The announcement is successfully displayed on the students' dashboards.

Backlog

Product Backlog

The **Product Backlog** is a prioritized list of all features, enhancements, and tasks that need to be developed over the course of the project. It includes high-level user stories and epics that describe the system's required functionalities.

The product backlog for **Campus Hub** may include the following **user stories** (examples):

1. Manage Student Records

- As an admin, I want to add, edit, and delete student records so that I can keep the database up to date.
- *Priority:* High

2. Track Attendance

- As a teacher, I want to mark attendance so that I can keep track of student presence.
- *Priority:* High

3. Fee Management

- As a student, I want to view my fee payment status so that I can manage my payments.

- *Priority: Medium*

4. **Instructor Dashboard**

- As a teacher, I want a dashboard to manage my courses, post assignments, and send announcements to students.
- *Priority: Medium*

5. **Course Management**

- As an admin, I want to allocate courses to students and teachers so that they can access course materials.
- *Priority: High*

The product backlog will evolve over time, with new items added or reprioritized based on feedback from stakeholders and the progress of the project.

Sprint Backlog

The **Sprint Backlog** is a subset of the product backlog that is selected for completion during a specific sprint. It is typically chosen during the sprint planning meeting and contains only the user stories or tasks that the team will work on during that sprint.

Example of a sprint backlog for **Sprint 1**:

- **Create Student Record CRUD Functionality**
 - Implement a form for admins to input new student records.
 - Add functionality for viewing, editing, and deleting student records.
 - *Status: Not Started*
- **Attendance Tracking**
 - Create a module for teachers to mark attendance.
 - Generate attendance reports for each student.
 - *Status: Not Started.*

Backlog management will be ongoing throughout the project. During each **Sprint Review** and **Retrospective**, the team will review what was completed, adjust the priorities of the product backlog, and plan for the next sprint.

Glossary

- **EMS:** Education Management System
- **CRUD:** Create, Read, Update, Delete
- **UI:** User Interface
- **SRS:** Software Requirement Specifications

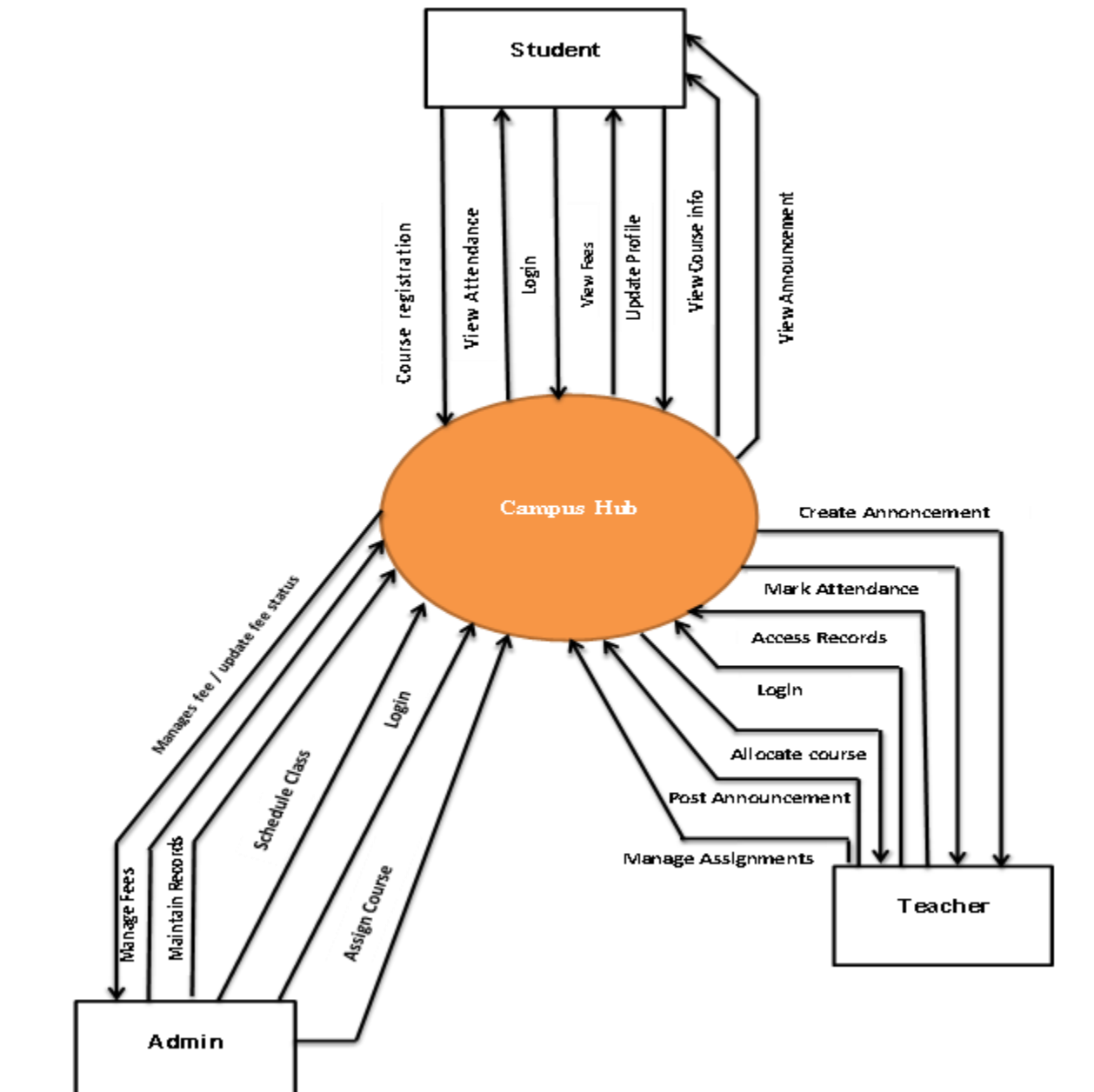
References

SRS Template:

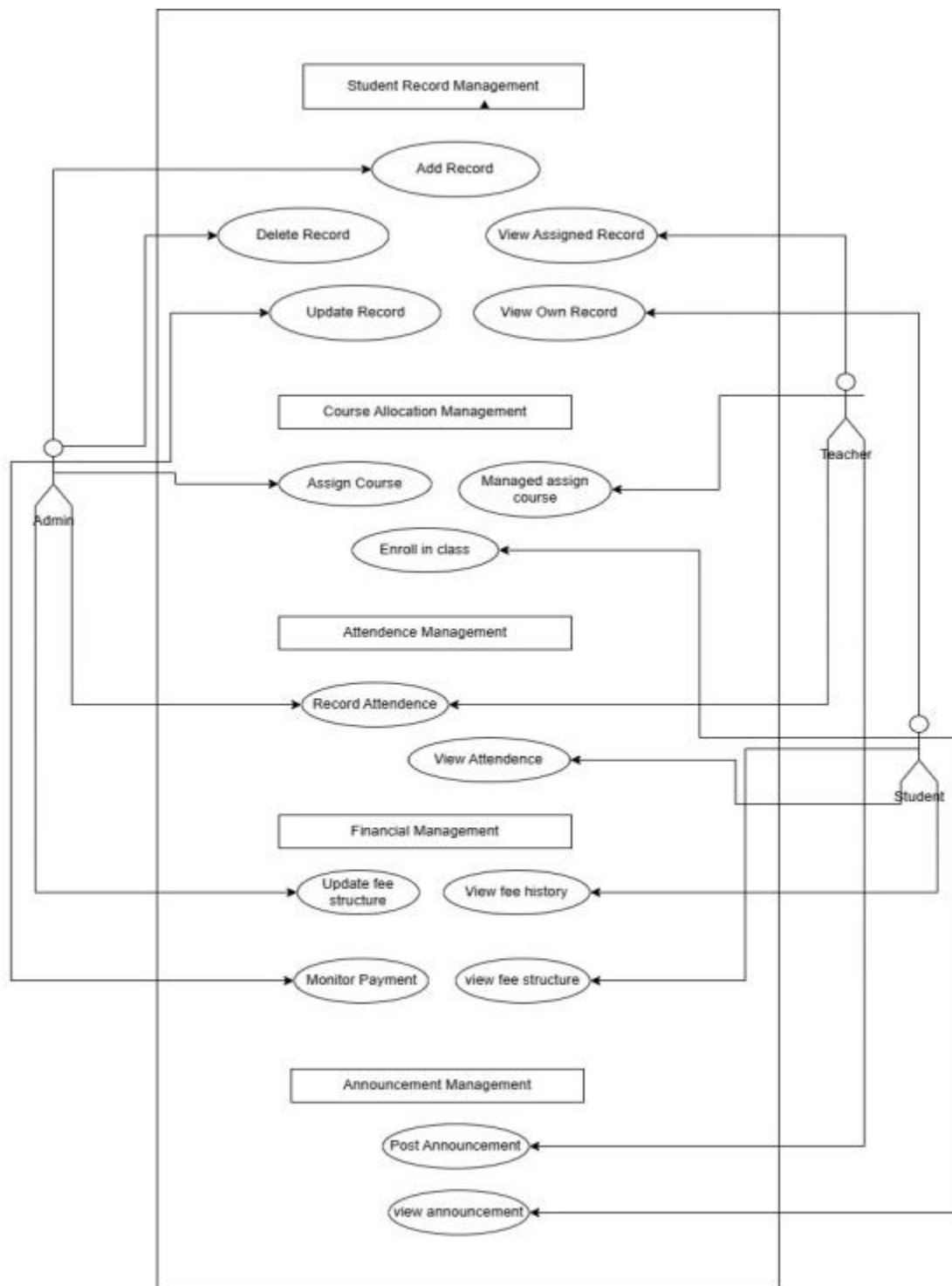
<https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf>

Diagrams

Context Diagram:

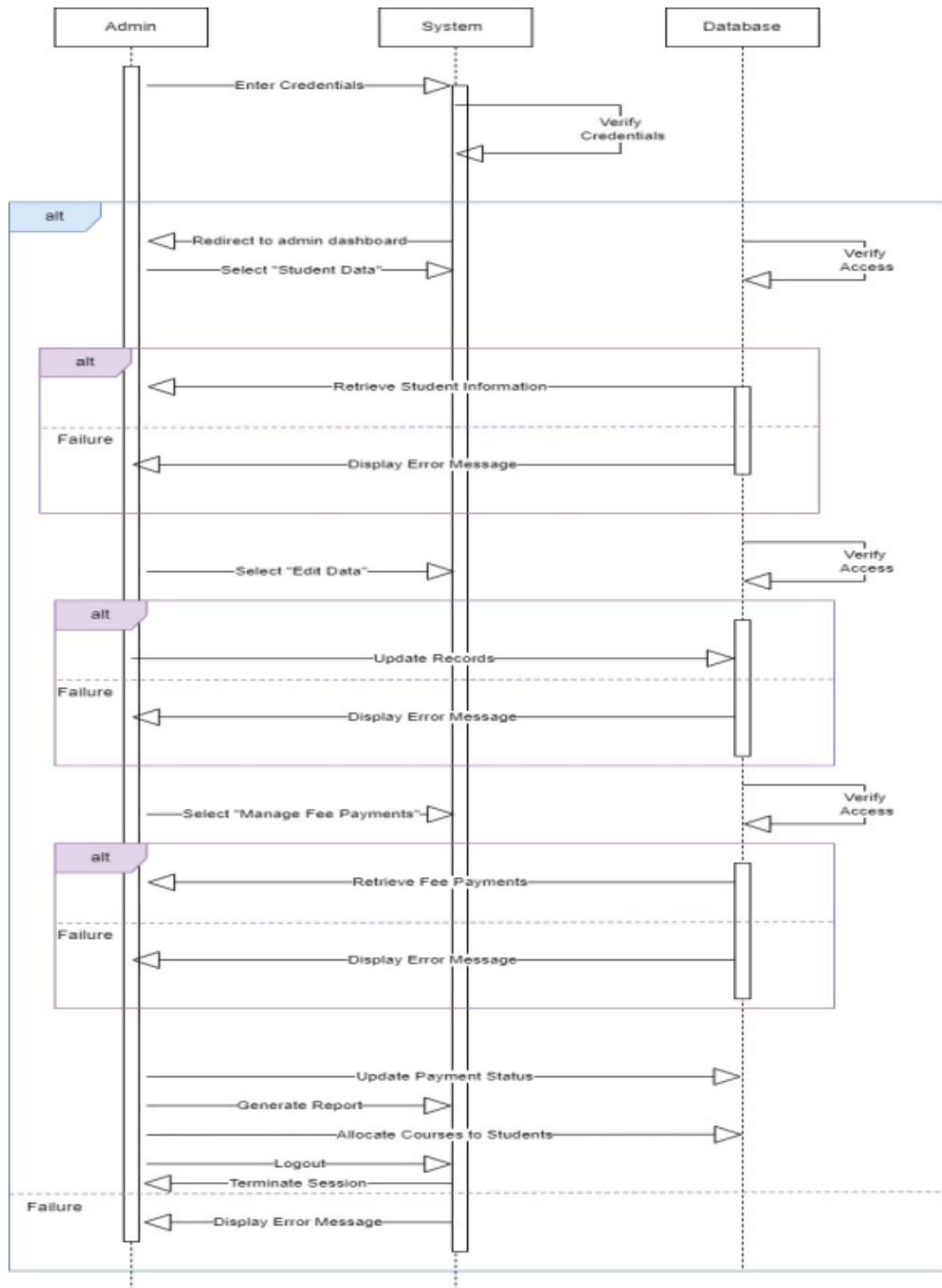


Use Case Diagram:

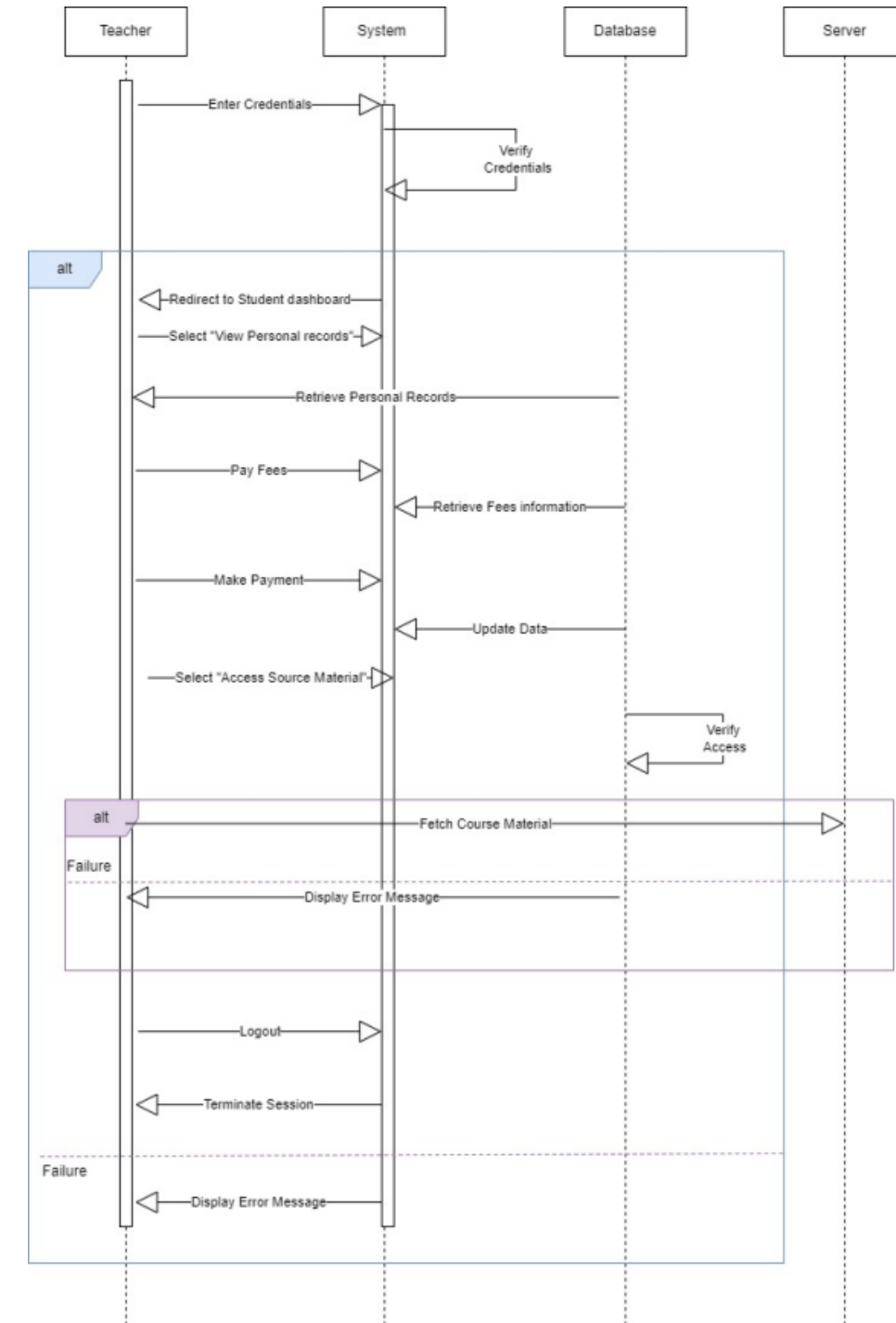


Sequence Diagram:

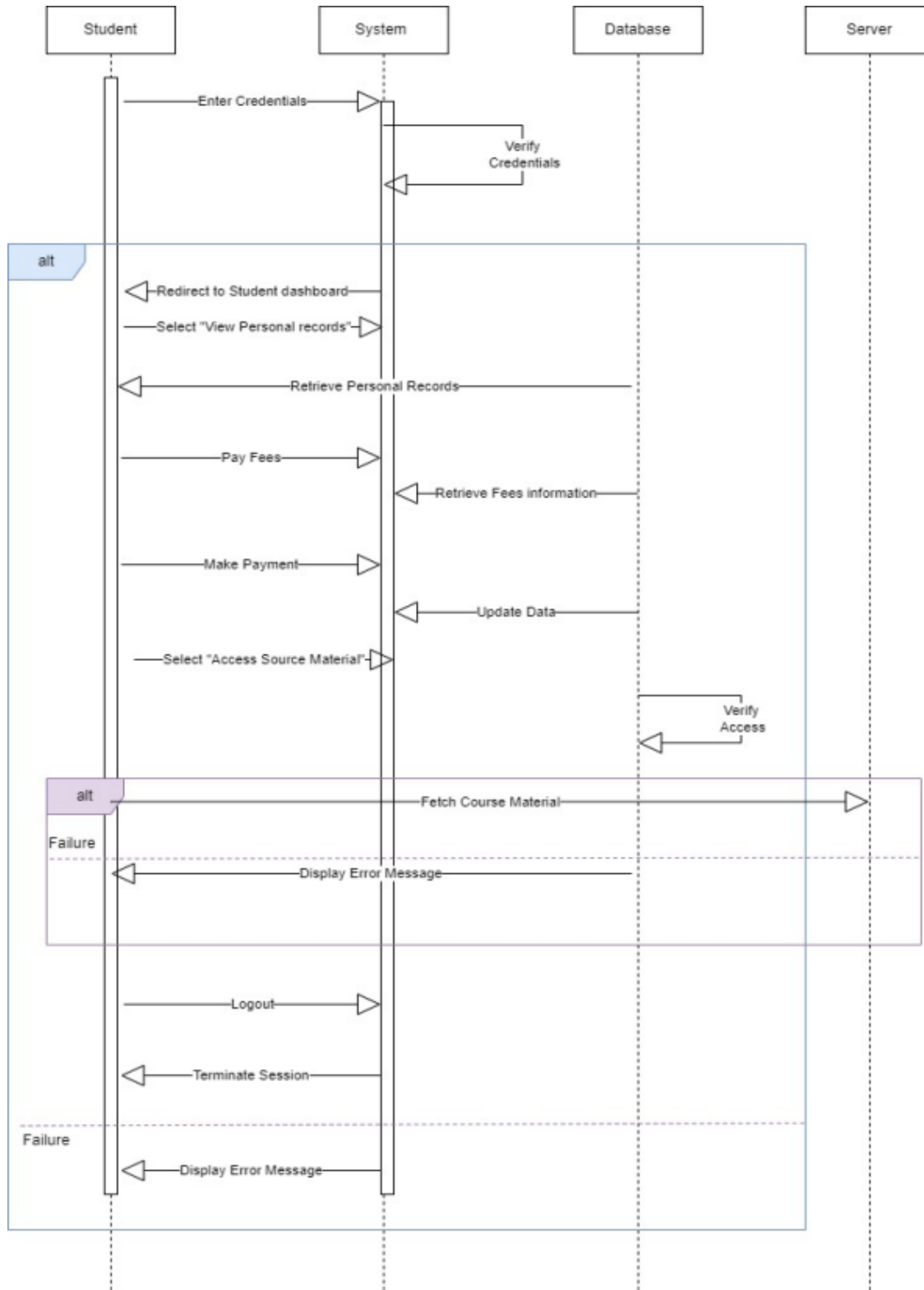
Admin



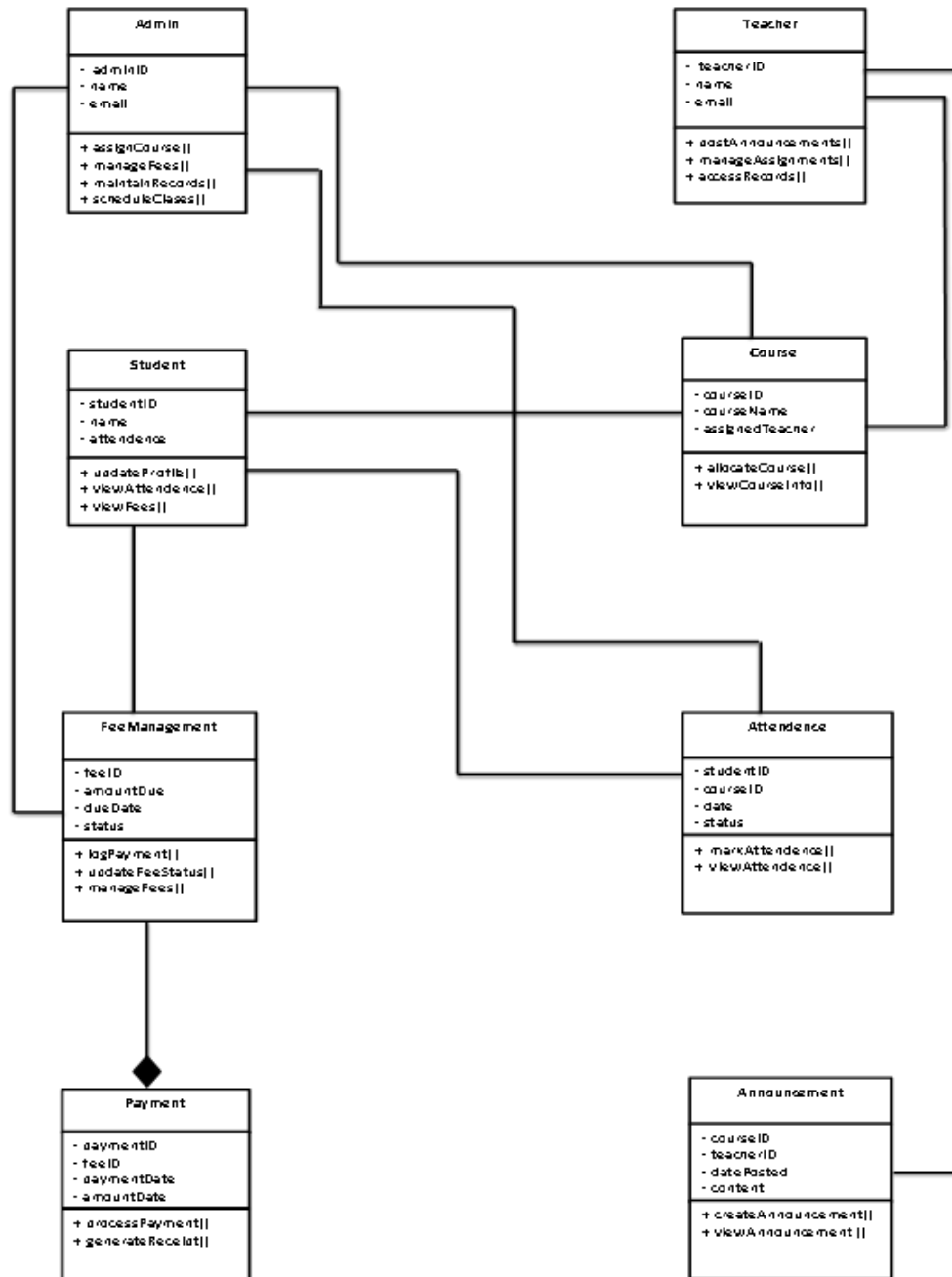
Teacher



Student



Class Diagram:



Architecture:

Client-Server Architecture with MVC Design Pattern

1. Primary Architecture: Client-Server Architecture

- **Definition:** The Client-Server architecture divides the application into two main components:
 - **Client:** The user interface that interacts with users (e.g., web browsers, mobile apps).
 - **Server:** The backend that processes asks, applies business logic, and manages data.
- **How it fits Campus Hub:**
 - **Client:** Web-based interface used by students, teachers, and administrators to interact with the system.
 - **Server:** Backend services built using Java, which handle business logic, communicate with the database, and serve the client requests.
 - **Database:** The data layer, which stores student records, attendance, and fee information.

Division of Responsibilities:

Aqsa Ahmed was responsible making the Context Diagram and Class Diagram.

Simaab Malik was responsible for making the Use Case diagram.

Hoor Spogmay took charge of creating the Sequence Diagram and finalizing the architecture. Compiled the document.

Implementation Phase :

Create Student Record CRUD Functionality

- o Implement a form for admins to input new student records.
- o Add functionality for viewing, editing, and deleting student records.
- o Status : Completed.

Display Student Data :

```
package main;

import java.io.BufferedReader;
import java.io.File;
import java.io.FileReader;
import java.io.IOException;
import static java.lang.System.exit;
import javax.swing.JOptionPane;

public class DisplayStudentData{
    public String readStudentInfo() {
        File file = new File("Studentdata.txt");
        StringBuilder sb = new StringBuilder();
        try {
            FileReader reader = new FileReader(file);
            BufferedReader breader = new BufferedReader(reader);
            try
            {
                String line;
```



```

        while ((line = breader.readLine()) != null) {
            sb.append(line).append("\n");
        }
    }

    finally
    {
        breader.close();
        reader.close();
    }
} catch (IOException e) {
    e.printStackTrace();

    JOptionPane.showMessageDialog(null, "Error occurred while reading student information.",
    "Error", JOptionPane.ERROR_MESSAGE);
}

return sb.toString();
}

public String readStudentCourses() {
    File file = new File("EnrolledCourses.txt");
    StringBuilder sb = new StringBuilder();
    try {
        FileReader reader = new FileReader(file);
        BufferedReader breader = new BufferedReader(reader);
        String line;
        while ((line = breader.readLine()) != null) {
            sb.append(line).append("\n");
        }
        breader.close();
        reader.close();
    }
}

```

```

    } catch (IOException e) {
        e.printStackTrace();
        JOptionPane.showMessageDialog(null, "Error occurred while reading student information.",
"Error", JOptionPane.ERROR_MESSAGE);
    }
    return sb.toString();
}
}

class DisplayStudentDataGUI extends javax.swing.JFrame {

    public DisplayStudentDataGUI() {
        initComponents();
    }

    // <editor-fold defaultstate="collapsed" desc="Generated Code"> //GEN-BEGIN: initComponents
    private void initComponents() {

        jPanel1 = new javax.swing.JPanel();
        logoutButton = new javax.swing.JButton();
        backButton = new javax.swing.JButton();
        PowerOffButton = new javax.swing.JButton();
        jLabel1 = new javax.swing.JLabel();
        courseButton = new javax.swing.JButton();
        displayStudentInfo = new javax.swing.JButton();
        jScrollPane1 = new javax.swing.JScrollPane();
        jTextArea1 = new javax.swing.JTextArea();

        setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);

```

```

jPanel1.setBackground(new java.awt.Color(255, 210, 255));

logoutButton.setBackground(new java.awt.Color(255, 204, 255));

logoutButton.setFont(new java.awt.Font("Lucida Sans Typewriter", 0, 14)); // NOI18N

logoutButton.setIcon(new
javax.swing.ImageIcon(getClass().getResource("/images/logout.png"))); // NOI18N

logoutButton.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        logoutButtonActionPerformed(evt);
    }
});

backButton.setBackground(new java.awt.Color(255, 204, 255));

backButton.setFont(new java.awt.Font("Lucida Sans Typewriter", 0, 14)); // NOI18N

backButton.setIcon(new javax.swing.ImageIcon(getClass().getResource("/images/back.png")));
// NOI18N

backButton.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        backButtonActionPerformed(evt);
    }
});

PowerOffButton.setBackground(new java.awt.Color(255, 204, 255));

PowerOffButton.setIcon(new javax.swing.ImageIcon(getClass().getResource("/images/off
.png"))); // NOI18N

PowerOffButton.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        PowerOffButtonActionPerformed(evt);
    }
});

```

```

    }
};

```

```

jLabel1.setFont(new java.awt.Font("Segoe Print", 0, 18)); // NOI18N
jLabel1.setText("Display Student Info");

```

```

courseButton.setBackground(new java.awt.Color(255, 204, 255));
courseButton.setText("Display Enrolled Courses");
courseButton.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        courseButtonActionPerformed(evt);
    }
});

```

```

displayStudentInfo.setBackground(new java.awt.Color(255, 204, 255));
displayStudentInfo.setText("Display Student Info");
displayStudentInfo.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        displayStudentInfoActionPerformed(evt);
    }
});

```

```

jTextArea1.setBackground(new java.awt.Color(255, 204, 255));
jTextArea1.setColumns(20);
jTextArea1.setRows(5);
jScrollPane1.setViewportView(jTextArea1);

```

```

javax.swing.GroupLayout jPanel1Layout = new javax.swing.GroupLayout(jPanel1);

```

```

jPanel1.setLayout(jPanel1Layout);
jPanel1Layout.setHorizontalGroup(
    jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
jPanel1Layout.createSequentialGroup()
            .addGap(0, 0, Short.MAX_VALUE)
            .addComponent(PowerOffButton, javax.swing.GroupLayout.PREFERRED_SIZE, 86,
javax.swing.GroupLayout.PREFERRED_SIZE))
        .addGroup(jPanel1Layout.createSequentialGroup()
            .addGap(21, 21, 21)

.addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(jPanel1Layout.createSequentialGroup()
        .addComponent(backButton)
        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        .addComponent(logoutButton, javax.swing.GroupLayout.PREFERRED_SIZE, 77,
javax.swing.GroupLayout.PREFERRED_SIZE))
        .addGroup(jPanel1Layout.createSequentialGroup()

.addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addComponent(jScrollPane1, javax.swing.GroupLayout.PREFERRED_SIZE, 442,
javax.swing.GroupLayout.PREFERRED_SIZE)

.addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING)
    .addComponent(jLabel1)
    .addGroup(jPanel1Layout.createSequentialGroup()
        .addComponent(displayStudentInfo)
        .addGap(18, 18, 18)
        .addComponent(courseButton))))
    .addContainerGap(28, Short.MAX_VALUE))))

```

```

);

jPanel1Layout.setVerticalGroup(
    jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(jPanel1Layout.createSequentialGroup())

        .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addComponent(PowerOffButton, javax.swing.GroupLayout.PREFERRED_SIZE, 82,
                javax.swing.GroupLayout.PREFERRED_SIZE)
            .addComponent(jLabel1, javax.swing.GroupLayout.Alignment.TRAILING))
            .addGap(18, 18, 18)

        .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
            .addComponent(displayStudentInfo, javax.swing.GroupLayout.PREFERRED_SIZE, 52,
                javax.swing.GroupLayout.PREFERRED_SIZE)
            .addComponent(courseButton, javax.swing.GroupLayout.PREFERRED_SIZE, 52,
                javax.swing.GroupLayout.PREFERRED_SIZE))
            .addGap(18, 18, 18)
            .addComponent(jScrollPane1, javax.swing.GroupLayout.DEFAULT_SIZE, 164,
                Short.MAX_VALUE)
            .addGap(65, 65, 65)

        .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addComponent(logoutButton, javax.swing.GroupLayout.PREFERRED_SIZE, 71,
                javax.swing.GroupLayout.PREFERRED_SIZE)
            .addComponent(backButton, javax.swing.GroupLayout.PREFERRED_SIZE, 71,
                javax.swing.GroupLayout.PREFERRED_SIZE))
            .addContainerGap())
);

javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
getContentPane().setLayout(layout);

```

```

        layout.setHorizontalGroup(
            layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                .addComponent(jPanel1, javax.swing.GroupLayout.DEFAULT_SIZE,
                    javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        );

        layout.setVerticalGroup(
            layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                .addComponent(jPanel1, javax.swing.GroupLayout.DEFAULT_SIZE,
                    javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        );

        pack();

        setLocationRelativeTo(null);
    } // </editor-fold> // GEN-END: initComponents

    private void logoutButtonActionPerformed(java.awt.event.ActionEvent evt) { // GEN-
FIRST:event_logoutButtonActionPerformed

        this.toBack();

        LoginPage login = new LoginPage();

        login.setVisible(true);

        login.toFront();
    } // GEN-LAST:event_logoutButtonActionPerformed

    private void backButtonActionPerformed(java.awt.event.ActionEvent evt) { // GEN-
FIRST:event_backButtonActionPerformed

        this.toBack();

    } // GEN-LAST:event_backButtonActionPerformed

```

```
private void PowerOffButtonActionPerformed(java.awt.event.ActionEvent evt) {GEN-
FIRST:event_PowerOffButtonActionPerformed
```

```
    exit(0);
```

```
}//GEN-LAST:event_PowerOffButtonActionPerformed
```

```
private void displayStudentInfoActionPerformed(java.awt.event.ActionEvent evt) {GEN-
FIRST:event_displayStudentInfoActionPerformed
```

```
    DisplayStudentData displayStudentData = new DisplayStudentData();
```

```
    String studentInfo = displayStudentData.readStudentInfo();
```

```
    JTextArea1.setText(studentInfo);
```

```
}//GEN-LAST:event_displayStudentInfoActionPerformed
```

```
private void courseButtonActionPerformed(java.awt.event.ActionEvent evt) {GEN-
FIRST:event_courseButtonActionPerformed
```

```
    DisplayStudentData displayStudentData = new DisplayStudentData();
```

```
    String studentInfo = displayStudentData.readStudentCourses();
```

```
    JTextArea1.setText(studentInfo);
```

```
}//GEN-LAST:event_courseButtonActionPerformed
```

```
// Variables declaration - do not modify//GEN-BEGIN:variables
```

```
private javax.swing.JButton PowerOffButton;
```

```
private javax.swing.JButton backButton;
```

```
private javax.swing.JButton courseButton;
```

```
private javax.swing.JButton displayStudentInfo;
```

```
private javax.swing.JLabel jLabel1;
```

```
private javax.swing.JPanel jPanel1;
```

```
private javax.swing.JScrollPane jScrollPane1;
```

```
private javax.swing.JTextArea jTextArea1;
```

```
private javax.swing.JButton logoutButton;
```



```
// End of variables declaration//GEN-END:variables  
}
```

2. Student Information :

```
package main;  
  
import java.io.BufferedWriter;  
import java.io.File;  
import java.io.*;  
import java.io.IOException;  
import static java.lang.System.exit;  
import javax.swing.JOptionPane;  
import java.util.ArrayList;  
  
public class Studentinfo {  
  
    //data members  
    private String name;  
    private String id;  
    private String age;  
    private String address;  
    private String personalContactNumber;  
    private String emergencyContactNumber;  
    private String gender;  
    private String nationality;  
    private String dob;  
    private String email;
```

```
//Constructor
public Studentinfo()
{
    name = null;
    id = null;
    age = null;
    address = null;
    personalContactNumber = null;
    emergencyContactNumber = null;
    gender = null;
    nationality = null;
    dob = null;
    email = null;
}

public Studentinfo(String name, String id, String age, String address, String
personalContactNumber, String emergencyContactNumber, String gender, String nationality, String
dob, String email)
{
    this.name = name;
    this.id = id;
    this.age = age;
    this.address = address;
    this.personalContactNumber = personalContactNumber;
    this.emergencyContactNumber = emergencyContactNumber;
    this.gender = gender;
    this.nationality = nationality;
    this.dob = dob;
    this.email = email;
```

```
}
```

```
// Getters
```

```
public String getName() {  
    return name;  
}
```

```
public String getId() {  
    return id;  
}
```

```
public String getAge() {  
    return age;  
}
```

```
public String getAddress() {  
    return address;  
}
```

```
public String getPersonalContactNumber() {  
    return personalContactNumber;  
}
```

```
public String getEmergencyContactNumber() {  
    return emergencyContactNumber;  
}
```

```
public String getGender() {  
    return gender;  
}
```

```
public String getNationality() {  
    return nationality;  
}
```

```
public String getDob() {  
    return dob;  
}
```

```
public String getEmail() {  
    return email;  
}
```

```
// Setters
```

```
public void setName(String name) {  
    this.name = name;  
}
```

```
public void setId(String id) {  
    this.id = id;  
}
```

```
public void setAge(String age) {  
    this.age = age;  
}
```

```
}
```

```
public void setAddress(String address) {  
    this.address = address;  
}
```

```
public void setPersonalContactNumber(String personalContactNumber) {  
    this.personalContactNumber = personalContactNumber;  
}
```

```
public void setEmergencyContactNumber(String emergencyContactNumber) {  
    this.emergencyContactNumber = emergencyContactNumber;  
}
```

```
public void setGender(String gender) {  
    this.gender = gender;  
}
```

```
public void setNationality(String nationality) {  
    this.nationality = nationality;  
}
```

```
public void setDob(String dob) {  
    this.dob = dob;  
}
```

```
public void setEmail(String email) {  
    this.email = email;
```

```
}
```

```
// Display info
```

```
@Override
```

```
public String toString() {
```

```
    return "Student Information\nName: " + name + "\n" +
```

```
        "ID: " + id + "\n" +
```

```
        "Age: " + age + "\n" +
```

```
        "Address: " + address + "\n" +
```

```
        "Personal Contact Number: " + personalContactNumber + "\n" +
```

```
        "Emergency Contact Number: " + emergencyContactNumber + "\n" +
```

```
        "Gender: " + gender + "\n" +
```

```
        "Nationality: " + nationality + "\n" +
```

```
        "Date of Birth: " + dob + "\n" +
```

```
        "Email: " + email + "\n";
```

```
}
```

```
}
```

```
class StudentinfoGUI extends javax.swing.JFrame {
```

```
/**
```

```
 * Creates new form StudentInfo
```

```
 */
```

```
public StudentinfoGUI() {
```

```
    initComponents();
```

```
}
```

```

private void saveStudentInfo() {
    File file = new File("Studentdata.txt");
    try {
        FileWriter writer = new FileWriter(file);
        BufferedWriter bwriter = new BufferedWriter(writer);
        try {
            String name = nameField.getText().trim();
            String id = idField.getText().trim();
            String age = ageField.getText().trim();
            String address = addressField.getText().trim();
            String personalContactNumber = personalContactNumberField.getText().trim();
            String emergencyContactNumber = emergencyContactNumberField.getText().trim();
            String gender = genderField.getText().trim();
            String nationality = nationalityField.getText().trim();
            String dob = dobField.getText().trim();
            String email = emailField.getText().trim();

            // Validate inputs
            if (name.isEmpty() || id.isEmpty() || age.isEmpty() || address.isEmpty() ||
                personalContactNumber.isEmpty() || emergencyContactNumber.isEmpty() ||
                gender.isEmpty() || nationality.isEmpty() || dob.isEmpty() || email.isEmpty()) {
                throw new IllegalArgumentException("All fields must be filled.");
            }
        }

        int ageInt;
        try {
            ageInt = Integer.parseInt(age);
            if (ageInt <= 0) {

```

```

        throw new IllegalArgumentException("Age must be a positive number.");
    }
} catch (NumberFormatException e) {
    throw new IllegalArgumentException("Age must be a valid number.", e);
}

if (!personalContactNumber.matches("\\d+")) {
    throw new IllegalArgumentException("Personal Contact Number must contain only digits.");
}

if (!emergencyContactNumber.matches("\\d+")) {
    throw new IllegalArgumentException("EmergencyContact Number must contain only digits.");
}

if (!email.matches("^([\\w.%+-]+@[\\w.-]+|\\.[a-zA-Z]{2,6}$)")) {
    throw new IllegalArgumentException("Email is not in a valid format.");
}

Studentinfo student = new Studentinfo(name, id, age, address, personalContactNumber,
emergencyContactNumber, gender, nationality, dob, email);

bwriter.write(student.toString());

JOptionPane.showMessageDialog(this, "Student Information saved to Studentdata.txt.");
} catch (IllegalArgumentException e) {
    JOptionPane.showMessageDialog(this, e.getMessage(), "Input Error",
JOptionPane.ERROR_MESSAGE);
} finally {
    try {
        bwriter.close();
    }
}

```



```

        writer.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}

} catch (IOException e) {
    e.printStackTrace();

    JOptionPane.showMessageDialog(this, "Error occurred while saving student information.", "Error",
    JOptionPane.ERROR_MESSAGE);
}

}

```

```
@SuppressWarnings("unchecked")
```

```
// <editor-fold defaultstate="collapsed" desc="Generated Code"> //GEN-BEGIN: initComponents
```

```
private void initComponents() {
```

```
    jButton1 = new javax.swing.JButton();
```

```
    jPanel1 = new javax.swing.JPanel();
```

```
    jLabel2 = new javax.swing.JLabel();
```

```
    jLabel3 = new javax.swing.JLabel();
```

```
    jLabel5 = new javax.swing.JLabel();
```

```
    jLabel6 = new javax.swing.JLabel();
```

```
    jLabel8 = new javax.swing.JLabel();
```

```
    jLabel9 = new javax.swing.JLabel();
```

```
    jLabel10 = new javax.swing.JLabel();
```

```
    jLabel11 = new javax.swing.JLabel();
```

```
jLabel4 = new javax.swing.JLabel();
jLabel7 = new javax.swing.JLabel();
jLabel1 = new javax.swing.JLabel();
saveButton = new javax.swing.JButton();
nameField = new javax.swing.JTextField();
idField = new javax.swing.JTextField();
ageField = new javax.swing.JTextField();
emailField = new javax.swing.JTextField();
nationalityField = new javax.swing.JTextField();
addressField = new javax.swing.JTextField();
personalContactNumberField = new javax.swing.JTextField();
dobField = new javax.swing.JTextField();
emergencyContactNumberField = new javax.swing.JTextField();
genderField = new javax.swing.JTextField();
genderComboBox = new javax.swing.JComboBox<>();
clearButton = new javax.swing.JButton();
logoutButton = new javax.swing.JButton();
jButton11 = new javax.swing.JButton();
jButton12 = new javax.swing.JButton();

jButton1.setText("jButton1");

setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);

jPanel1.setBackground(new java.awt.Color(255, 210, 255));
jPanel1.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());

jLabel2.setBackground(new java.awt.Color(255, 204, 255));
```

```
jLabel2.setFont(new java.awt.Font("Lucida Sans", 0, 12)); // NOI18N
jLabel2.setText("Email");
jPanel1.add(jLabel2, new org.netbeans.lib.awtextra.AbsoluteConstraints(37, 358, 80, 30));

jLabel3.setBackground(new java.awt.Color(255, 204, 255));
jLabel3.setFont(new java.awt.Font("Lucida Sans", 0, 12)); // NOI18N
jLabel3.setText("Nationality");
jPanel1.add(jLabel3, new org.netbeans.lib.awtextra.AbsoluteConstraints(339, 124, 80, 30));

jLabel5.setBackground(new java.awt.Color(255, 204, 255));
jLabel5.setFont(new java.awt.Font("Lucida Sans", 0, 12)); // NOI18N
jLabel5.setText("Date Of Birth");
jLabel5.setMaximumSize(new java.awt.Dimension(80, 30));
jPanel1.add(jLabel5, new org.netbeans.lib.awtextra.AbsoluteConstraints(339, 242, 80, 30));

jLabel6.setBackground(new java.awt.Color(255, 204, 255));
jLabel6.setFont(new java.awt.Font("Lucida Sans", 0, 12)); // NOI18N
jLabel6.setText("Address");
jPanel1.add(jLabel6, new org.netbeans.lib.awtextra.AbsoluteConstraints(339, 180, 80, 30));

jLabel8.setBackground(new java.awt.Color(255, 204, 255));
jLabel8.setFont(new java.awt.Font("Lucida Sans", 0, 12)); // NOI18N
jLabel8.setText("Age");
jPanel1.add(jLabel8, new org.netbeans.lib.awtextra.AbsoluteConstraints(31, 242, 80, 30));

jLabel9.setBackground(new java.awt.Color(255, 204, 255));
jLabel9.setFont(new java.awt.Font("Lucida Sans", 0, 12)); // NOI18N
jLabel9.setText("Emergency Contact Number");
```

```
jPanel1.add(jLabel9, new org.netbeans.lib.awtextra.AbsoluteConstraints(339, 358, 165, 30));
```

```
jLabel10.setBackground(new java.awt.Color(255, 204, 255));
```

```
jLabel10.setFont(new java.awt.Font("Lucida Sans", 0, 12)); // NOI18N
```

```
jLabel10.setText("ID");
```

```
jPanel1.add(jLabel10, new org.netbeans.lib.awtextra.AbsoluteConstraints(31, 180, 80, 30));
```

```
jLabel11.setBackground(new java.awt.Color(255, 204, 255));
```

```
jLabel11.setFont(new java.awt.Font("Lucida Sans", 0, 12)); // NOI18N
```

```
jLabel11.setText("Name");
```

```
jPanel1.add(jLabel11, new org.netbeans.lib.awtextra.AbsoluteConstraints(33, 124, 80, 30));
```

```
jLabel4.setBackground(new java.awt.Color(255, 204, 255));
```

```
jLabel4.setFont(new java.awt.Font("Lucida Sans", 0, 12)); // NOI18N
```

```
jLabel4.setText("Gender");
```

```
jLabel4.setPreferredSize(new java.awt.Dimension(80, 30));
```

```
jPanel1.add(jLabel4, new org.netbeans.lib.awtextra.AbsoluteConstraints(31, 296, -1, -1));
```

```
jLabel7.setBackground(new java.awt.Color(255, 204, 255));
```

```
jLabel7.setFont(new java.awt.Font("Lucida Sans", 0, 12)); // NOI18N
```

```
jLabel7.setText("Personal Contact Number");
```

```
jPanel1.add(jLabel7, new org.netbeans.lib.awtextra.AbsoluteConstraints(339, 296, 155, 30));
```

```
jLabel1.setFont(new java.awt.Font("Segoe Print", 1, 18)); // NOI18N
```

```
jLabel1.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
```

```
jLabel1.setText("Enter Your Particulars");
```

```
jPanel1.add(jLabel1, new org.netbeans.lib.awtextra.AbsoluteConstraints(230, 40, 275, 34));
```

```

saveButton.setBackground(new java.awt.Color(255, 204, 255));
saveButton.setFont(new java.awt.Font("Dialog", 0, 14)); // NOI18N
saveButton.setIcon(new javax.swing.ImageIcon(getClass().getResource("/images/save.png")));
// NOI18N

saveButton.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        saveButtonActionPerformed(evt);
    }
});

jPanel1.add(saveButton, new org.netbeans.lib.awtextra.AbsoluteConstraints(240, 430, 90, 90));

```

```

nameField.setBackground(new java.awt.Color(255, 204, 255));
nameField.setFont(new java.awt.Font("Lucida Sans", 0, 12)); // NOI18N
nameField.setText("Enter your Name");
nameField.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        nameFieldActionPerformed(evt);
    }
});

jPanel1.add(nameField, new org.netbeans.lib.awtextra.AbsoluteConstraints(125, 117, 179,
44));

```

```

idField.setBackground(new java.awt.Color(255, 204, 255));
idField.setFont(new java.awt.Font("Lucida Sans", 0, 12)); // NOI18N
idField.setText("Enter your ID");
idField.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        idFieldActionPerformed(evt);
    }
}

```

```
});
```

```
jPanel1.add(idField, new org.netbeans.lib.awtextra.AbsoluteConstraints(123, 173, 179, 44));
```

```
ageField.setBackground(new java.awt.Color(255, 204, 255));
```

```
ageField.setFont(new java.awt.Font("Lucida Sans", 0, 12)); // NOI18N
```

```
ageField.setText("Enter your Age");
```

```
ageField.addActionListener(new java.awt.event.ActionListener() {
```

```
    public void actionPerformed(java.awt.event.ActionEvent evt) {
```

```
        ageFieldActionPerformed(evt);
```

```
    }
```

```
});
```

```
jPanel1.add(ageField, new org.netbeans.lib.awtextra.AbsoluteConstraints(123, 235, 179, 44));
```

```
emailField.setBackground(new java.awt.Color(255, 204, 255));
```

```
emailField.setFont(new java.awt.Font("Lucida Sans", 0, 12)); // NOI18N
```

```
emailField.setText("Enter your Email");
```

```
emailField.addActionListener(new java.awt.event.ActionListener() {
```

```
    public void actionPerformed(java.awt.event.ActionEvent evt) {
```

```
        emailFieldActionPerformed(evt);
```

```
    }
```

```
});
```

```
jPanel1.add(emailField, new org.netbeans.lib.awtextra.AbsoluteConstraints(123, 351, 179, 44));
```

```
nationalityField.setBackground(new java.awt.Color(255, 204, 255));
```

```
nationalityField.setText("Enter your Nationality");
```

```
nationalityField.addActionListener(new java.awt.event.ActionListener() {
```

```
    public void actionPerformed(java.awt.event.ActionEvent evt) {
```

```
        nationalityFieldActionPerformed(evt);
```

```

    }
    });

    jPanel1.add(nationalityField, new org.netbeans.lib.awtextra.AbsoluteConstraints(512, 117, 177,
44));

    addressField.setBackground(new java.awt.Color(255, 204, 255));
    addressField.setText("Enter your Address");
    addressField.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            addressFieldActionPerformed(evt);
        }
    });
    jPanel1.add(addressField, new org.netbeans.lib.awtextra.AbsoluteConstraints(512, 173, 177,
44));

    personalContactNumberField.setBackground(new java.awt.Color(255, 204, 255));
    personalContactNumberField.setText("Enter your Contact");
    personalContactNumberField.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            personalContactNumberFieldActionPerformed(evt);
        }
    });
    jPanel1.add(personalContactNumberField, new
org.netbeans.lib.awtextra.AbsoluteConstraints(512, 289, 177, 44));

    dobField.setBackground(new java.awt.Color(255, 204, 255));
    dobField.setText("Enter your DOB");
    dobField.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {

```

```

        dobFieldActionPerformed(evt);
    }
});

jPanel1.add(dobField, new org.netbeans.lib.awtextra.AbsoluteConstraints(512, 235, 177, 44));

emergencyContactNumberField.setBackground(new java.awt.Color(255, 204, 255));
emergencyContactNumberField.setText("Enter your Contact");
emergencyContactNumberField.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        emergencyContactNumberFieldActionPerformed(evt);
    }
});

jPanel1.add(emergencyContactNumberField, new
org.netbeans.lib.awtextra.AbsoluteConstraints(510, 350, 177, 44));

genderField.setBackground(new java.awt.Color(255, 204, 255));
genderField.setFont(new java.awt.Font("Lucida Sans", 0, 12)); // NOI18N
genderField.setText("Enter your Gender");
genderField.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        genderFieldActionPerformed(evt);
    }
});

jPanel1.add(genderField, new org.netbeans.lib.awtextra.AbsoluteConstraints(123, 289, 89,
44));

genderComboBox.setBackground(new java.awt.Color(255, 204, 255));
genderComboBox.setFont(new java.awt.Font("Lucida Sans", 0, 12)); // NOI18N

```



```
genderComboBox.setModel(new javax.swing.DefaultComboBoxModel<>(new String[] { "male",
"female" }));
```

```
genderComboBox.setCursor(new java.awt.Cursor(java.awt.Cursor.DEFAULT_CURSOR));
```

```
genderComboBox.addActionListener(new java.awt.event.ActionListener() {
```

```
    public void actionPerformed(java.awt.event.ActionEvent evt) {
```

```
        genderComboBoxActionPerformed(evt);
```

```
    }
```

```
});
```

```
jPanel1.add(genderComboBox, new org.netbeans.lib.awtextra.AbsoluteConstraints(224, 293,
78, 34));
```

```
clearButton.setBackground(new java.awt.Color(255, 204, 255));
```

```
clearButton.setFont(new java.awt.Font("Lucida Sans Typewriter", 0, 14)); // NOI18N
```

```
clearButton.setIcon(new
javax.swing.ImageIcon(getClass().getResource("/images/reset.png"))); // NOI18N
```

```
clearButton.addActionListener(new java.awt.event.ActionListener() {
```

```
    public void actionPerformed(java.awt.event.ActionEvent evt) {
```

```
        clearButtonActionPerformed(evt);
```

```
    }
```

```
});
```

```
jPanel1.add(clearButton, new org.netbeans.lib.awtextra.AbsoluteConstraints(370, 430, 90,
90));
```

```
logoutButton.setBackground(new java.awt.Color(255, 204, 255));
```

```
logoutButton.setIcon(new
javax.swing.ImageIcon(getClass().getResource("/images/logout.png"))); // NOI18N
```

```
logoutButton.addActionListener(new java.awt.event.ActionListener() {
```

```
    public void actionPerformed(java.awt.event.ActionEvent evt) {
```

```
        logoutButtonActionPerformed(evt);
```

```
    }
```

```

});

jPanel1.add(loginButton, new org.netbeans.lib.awtextra.AbsoluteConstraints(610, 480, 90,
90));

jButton11.setBackground(new java.awt.Color(255, 204, 255));

jButton11.setIcon(new javax.swing.ImageIcon(getClass().getResource("/images/off.png"))); //
NOI18N

jButton11.addActionListener(new java.awt.event.ActionListener() {

    public void actionPerformed(java.awt.event.ActionEvent evt) {

        jButton11ActionPerformed(evt);

    }

});

jPanel1.add(jButton11, new org.netbeans.lib.awtextra.AbsoluteConstraints(600, 0, 90, 90));

jButton12.setBackground(new java.awt.Color(255, 204, 255));

jButton12.setIcon(new javax.swing.ImageIcon(getClass().getResource("/images/back.png"))); //
NOI18N

jButton12.addActionListener(new java.awt.event.ActionListener() {

    public void actionPerformed(java.awt.event.ActionEvent evt) {

        jButton12ActionPerformed(evt);

    }

});

jPanel1.add(jButton12, new org.netbeans.lib.awtextra.AbsoluteConstraints(20, 490, 90, 90));

javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
getContentPane().setLayout(layout);
layout.setHorizontalGroup(

    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

        .addComponent(jPanel1, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)

```

```

);

layout.setVerticalGroup(
    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(layout.createSequentialGroup()
            .addContainerGap()
            .addComponent(jPanel1, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))
        );

pack();

setLocationRelativeTo(null);

} // </editor-fold> // GEN-END: initComponents

private void nameFieldActionPerformed(java.awt.event.ActionEvent evt) { // GEN-
FIRST:event_nameFieldActionPerformed

    ArrayList<String> Student = new ArrayList<>();

    Student.add(nameField.getText());

} // GEN-LAST:event_nameFieldActionPerformed

private void idFieldActionPerformed(java.awt.event.ActionEvent evt) { // GEN-
FIRST:event_idFieldActionPerformed

    // TODO add your handling code here:

} // GEN-LAST:event_idFieldActionPerformed

private void ageFieldActionPerformed(java.awt.event.ActionEvent evt) { // GEN-
FIRST:event_ageFieldActionPerformed

    // TODO add your handling code here:

} // GEN-LAST:event_ageFieldActionPerformed

```

```
private void emailFieldActionPerformed(java.awt.event.ActionEvent evt) { //GEN-FIRST:event_emailFieldActionPerformed
```

```
    // TODO add your handling code here:
```

```
    } //GEN-LAST:event_emailFieldActionPerformed
```

```
private void nationalityFieldActionPerformed(java.awt.event.ActionEvent evt) { //GEN-FIRST:event_nationalityFieldActionPerformed
```

```
    // TODO add your handling code here:
```

```
    } //GEN-LAST:event_nationalityFieldActionPerformed
```

```
private void addressFieldActionPerformed(java.awt.event.ActionEvent evt) { //GEN-FIRST:event_addressFieldActionPerformed
```

```
    // TODO add your handling code here:
```

```
    } //GEN-LAST:event_addressFieldActionPerformed
```

```
private void personalContactNumberFieldActionPerformed(java.awt.event.ActionEvent evt) { //GEN-FIRST:event_personalContactNumberFieldActionPerformed
```

```
    // TODO add your handling code here:
```

```
    } //GEN-LAST:event_personalContactNumberFieldActionPerformed
```

```
private void dobFieldActionPerformed(java.awt.event.ActionEvent evt) { //GEN-FIRST:event_dobFieldActionPerformed
```

```
    // TODO add your handling code here:
```

```
    } //GEN-LAST:event_dobFieldActionPerformed
```

```
private void emergencyContactNumberFieldActionPerformed(java.awt.event.ActionEvent evt) { //GEN-FIRST:event_emergencyContactNumberFieldActionPerformed
```

```
    // TODO add your handling code here:
```

```
    } //GEN-LAST:event_emergencyContactNumberFieldActionPerformed
```

```
private void genderFieldActionPerformed(java.awt.event.ActionEvent evt) {GEN-
FIRST:event_genderFieldActionPerformed
```

```
}//GEN-LAST:event_genderFieldActionPerformed
```

```
private void saveButtonActionPerformed(java.awt.event.ActionEvent evt) {GEN-
FIRST:event_saveButtonActionPerformed
```

```
    saveStudentInfo();
```

```
}//GEN-LAST:event_saveButtonActionPerformed
```

```
private void genderComboBoxActionPerformed(java.awt.event.ActionEvent evt) {GEN-
FIRST:event_genderComboBoxActionPerformed
```

```
    genderField.setText(genderComboBox.getSelectedItem().toString());
```

```
    genderField.setEditable(false);
```

```
}//GEN-LAST:event_genderComboBoxActionPerformed
```

```
private void clearButtonActionPerformed(java.awt.event.ActionEvent evt) {GEN-
FIRST:event_clearButtonActionPerformed
```

```
    nameField.setText("");
```

```
    idField.setText("");
```

```
    ageField.setText("");
```

```
    addressField.setText("");
```

```
    personalContactNumberField.setText("");
```

```
    emergencyContactNumberField.setText("");
```

```
    genderComboBox.setSelectedIndex(0);
```

```
    nationalityField.setText("");
```

```
    dobField.setText("");
```

```
    emailField.setText("");
```

```
}//GEN-LAST:event_clearButtonActionPerformed
```

```
private void logoutButtonActionPerformed(java.awt.event.ActionEvent evt) {//GEN-FIRST:event_logoutButtonActionPerformed
```

```
    this.toBack();
```

```
    LoginPage login = new LoginPage();
```

```
    login.setVisible(true);
```

```
    loginToFront();
```

```
//GEN-LAST:event_logoutButtonActionPerformed
```

```
private void jButton11ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-FIRST:event_jButton11ActionPerformed
```

```
    exit(0);
```

```
//GEN-LAST:event_jButton11ActionPerformed
```

```
private void jButton12ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-FIRST:event_jButton12ActionPerformed
```

```
    this.toBack();
```

```
    Student student = new Student();
```

```
    student.setVisible(true);
```

```
    studentToFront();
```

```
//GEN-LAST:event_jButton12ActionPerformed
```

```
// Variables declaration - do not modify//GEN-BEGIN:variables
```

```
private javax.swing.JTextField addressField;
```

```
private javax.swing.JTextField ageField;
```

```
private javax.swing.JButton clearButton;
```

```
private javax.swing.JTextField dobField;
```

```
private javax.swing.JTextField emailField;
```

```
private javax.swing.JTextField emergencyContactNumberField;
```

```
private javax.swing.JComboBox<String> genderComboBox;
```

```

private javax.swing.JTextField genderField;
private javax.swing.JTextField idField;
private javax.swing.JButton jButton1;
private javax.swing.JButton jButton11;
private javax.swing.JButton jButton12;
private javax.swing.JLabel jLabel1;
private javax.swing.JLabel jLabel10;
private javax.swing.JLabel jLabel11;
private javax.swing.JLabel jLabel2;
private javax.swing.JLabel jLabel3;
private javax.swing.JLabel jLabel4;
private javax.swing.JLabel jLabel5;
private javax.swing.JLabel jLabel6;
private javax.swing.JLabel jLabel7;
private javax.swing.JLabel jLabel8;
private javax.swing.JLabel jLabel9;
private javax.swing.JPanel jPanel1;
private javax.swing.JButton logoutButton;
private javax.swing.JTextField nameField;
private javax.swing.JTextField nationalityField;
private javax.swing.JTextField personalContactNumberField;
private javax.swing.JButton saveButton;

// End of variables declaration//GEN-END:variables
}

```

3. Edit Student Information:

```
package main;
```

```

import java.awt.Color;
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import static java.lang.System.exit;
import java.util.ArrayList;
import javax.swing.JOptionPane;
/**
 *
 * @author hoors
 */
public class editStudentinfo
{
    public void editName()
    {
        try {
            // Read existing student data from the file
            BufferedReader reader = new BufferedReader(new FileReader("Studentdata.txt"));
            ArrayList<String> lines = new ArrayList<>();
            String line;
            while ((line = reader.readLine()) != null) {
                lines.add(line);
            }
            reader.close();

            // Find the line containing the name and update it

```



```

for (int i = 0; i < lines.size(); i++) {
    if (lines.get(i).startsWith("Name:")) {
        // Prompt the user for the new name

        String newName = JOptionPane.showInputDialog("Enter the new name:");

        if (newName == null) {
            return;
        }

        lines.set(i, "Name: " + newName);

        break; // Once we found and updated the name, we can stop searching
    }
}

// Write the modified data back to the file
BufferedWriter writer = new BufferedWriter(new FileWriter("Studentdata.txt"));
for (String updatedLine : lines) {
    writer.write(updatedLine);
    writer.newLine();
}
writer.close();

JOptionPane.showMessageDialog(null, "Name updated successfully!");
} catch (IOException e) {
    e.printStackTrace();

    JOptionPane.showMessageDialog(null, "Error occurred while updating name.", "Error",
JOptionPane.ERROR_MESSAGE);
}
}

```

```

public void editdob()
{
    try {
        // Read existing student data from the file
        BufferedReader reader = new BufferedReader(new FileReader("Studentdata.txt"));
        ArrayList<String> lines = new ArrayList<>();
        String line;
        while ((line = reader.readLine()) != null) {
            lines.add(line);
        }
        reader.close();

        // Find the line containing the name and update it
        for (int i = 0; i < lines.size(); i++) {
            if (lines.get(i).startsWith("Date of Birth:")) {
                // Prompt the user for the new name
                String newDob = JOptionPane.showInputDialog("Enter the new Date of birth:");
                if (newDob == null) {
                    return;
                }
                lines.set(i, "Date of Birth: " + newDob);
                break; // Once we found and updated the name, we can stop searching
            }
        }

        // Write the modified data back to the file
        BufferedWriter writer = new BufferedWriter(new FileWriter("Studentdata.txt"));
        for (String updatedLine : lines) {

```

```

        writer.write(updatedLine);

        writer.newLine();
    }

    writer.close();

    JOptionPane.showMessageDialog(null, "Date of Birth updated successfully!");
} catch (IOException e) {
    e.printStackTrace();

    JOptionPane.showMessageDialog(null, "Error occurred while updating Date of Birth.", "Error",
JOptionPane.ERROR_MESSAGE);
}
}

public void editAddress()
{
    try {
        // Read existing student data from the file

        BufferedReader reader = new BufferedReader(new FileReader("Studentdata.txt"));

        ArrayList<String> lines = new ArrayList<>();

        String line;

        while ((line = reader.readLine()) != null) {

            lines.add(line);
        }

        reader.close();

        // Find the line containing the name and update it
        for (int i = 0; i < lines.size(); i++) {
            if (lines.get(i).startsWith("Address:")) {

```

```

        // Prompt the user for the new name
        String newAddress = JOptionPane.showInputDialog("Enter the new Address:");
        if (newAddress == null) {
            return;
        }
        lines.set(i, "Address: " + newAddress);
        break; // Once we found and updated the name, we can stop searching
    }
}

// Write the modified data back to the file
BufferedWriter writer = new BufferedWriter(new FileWriter("Studentdata.txt"));
for (String updatedLine : lines) {
    writer.write(updatedLine);
    writer.newLine();
}
writer.close();

JOptionPane.showMessageDialog(null, "Address updated successfully!");
} catch (IOException e) {
    e.printStackTrace();

    JOptionPane.showMessageDialog(null, "Error occurred while updating Address.", "Error",
JOptionPane.ERROR_MESSAGE);
}
}

public void editEmail()
{

```

```

try {
    // Read existing student data from the file
    BufferedReader reader = new BufferedReader(new FileReader("Studentdata.txt"));
    ArrayList<String> lines = new ArrayList<>();
    String line;
    while ((line = reader.readLine()) != null) {
        lines.add(line);
    }
    reader.close();

    // Find the line containing the name and update it
    for (int i = 0; i < lines.size(); i++) {
        if (lines.get(i).startsWith("Email:")) {
            // Prompt the user for the new name
            String newEmail = JOptionPane.showInputDialog("Enter the new Email:");
            if (newEmail == null) {
                return;
            }
            lines.set(i, "Email: " + newEmail);
            break; // Once we found and updated the name, we can stop searching
        }
    }

    // Write the modified data back to the file
    BufferedWriter writer = new BufferedWriter(new FileWriter("Studentdata.txt"));
    for (String updatedLine : lines) {
        writer.write(updatedLine);
        writer.newLine();
    }
}

```

```

    }

    writer.close();

    JOptionPane.showMessageDialog(null, "Email updated successfully!");
} catch (IOException e) {
    e.printStackTrace();

    JOptionPane.showMessageDialog(null, "Error occurred while updating Email.", "Error",
JOptionPane.ERROR_MESSAGE);
}
}

public void editID()
{
    try {
        // Read existing student data from the file

        BufferedReader reader = new BufferedReader(new FileReader("Studentdata.txt"));
        ArrayList<String> lines = new ArrayList<>();

        String line;
        while ((line = reader.readLine()) != null) {
            lines.add(line);
        }

        reader.close();

        // Find the line containing the name and update it
        for (int i = 0; i < lines.size(); i++) {
            if (lines.get(i).startsWith("ID:")) {
                // Prompt the user for the new name

                String newID = JOptionPane.showInputDialog("Enter the new ID:");

```

```

        if (newID == null) {
            return;
        }
        lines.set(i, "ID: " + newID);
        break; // Once we found and updated the name, we can stop searching
    }
}

// Write the modified data back to the file
BufferedWriter writer = new BufferedWriter(new FileWriter("Studentdata.txt"));
for (String updatedLine : lines) {
    writer.write(updatedLine);
    writer.newLine();
}
writer.close();

JOptionPane.showMessageDialog(null, "ID updated successfully!");
} catch (IOException e) {
    e.printStackTrace();
    JOptionPane.showMessageDialog(null, "Error occurred while updating ID.", "Error",
JOptionPane.ERROR_MESSAGE);
}
}

public void editAge()
{
    try {
        // Read existing student data from the file
        BufferedReader reader = new BufferedReader(new FileReader("Studentdata.txt"));

```

```

ArrayList<String> lines = new ArrayList<>();

String line;

while ((line = reader.readLine()) != null) {

    lines.add(line);

}

reader.close();


// Find the line containing the name and update it
for (int i = 0; i < lines.size(); i++) {

    if (lines.get(i).startsWith("Age:")) {

        // Prompt the user for the new name

        String newAge = JOptionPane.showInputDialog("Enter the new Age:");

        if (newAge == null) {

            return;

        }

        lines.set(i, "Age: " + newAge);

        break; // Once we found and updated the name, we can stop searching

    }

}


// Write the modified data back to the file

BufferedWriter writer = new BufferedWriter(new FileWriter("Studentdata.txt"));

for (String updatedLine : lines) {

    writer.write(updatedLine);

    writer.newLine();

}

writer.close();

```



```

        JOptionPane.showMessageDialog(null, "Age updated successfully!");
    } catch (IOException e) {
        e.printStackTrace();

        JOptionPane.showMessageDialog(null, "Error occurred while updating Age.", "Error",
JOptionPane.ERROR_MESSAGE);
    }
}

```

```

public void editGender()
{
    try {
        // Read existing student data from the file
        BufferedReader reader = new BufferedReader(new FileReader("Studentdata.txt"));
        ArrayList<String> lines = new ArrayList<>();
        String line;
        while ((line = reader.readLine()) != null) {
            lines.add(line);
        }
        reader.close();

        // Find the line containing the name and update it
        for (int i = 0; i < lines.size(); i++) {
            if (lines.get(i).startsWith("Gender: ")) {
                // Prompt the user for the new name
                String newGender = JOptionPane.showInputDialog("Enter the new Gender:");
                if (newGender == null) {
                    return;
                }
            }
        }
    }
}

```

```

    }

    lines.set(i, "Gender: " + newGender);

    break; // Once we found and updated the name, we can stop searching
}

}

// Write the modified data back to the file
BufferedWriter writer = new BufferedWriter(new FileWriter("Studentdata.txt"));
for (String updatedLine : lines) {
    writer.write(updatedLine);
    writer.newLine();
}
writer.close();

JOptionPane.showMessageDialog(null, "Gender updated successfully!");
} catch (IOException e) {
    e.printStackTrace();

    JOptionPane.showMessageDialog(null, "Error occurred while updating Gender.", "Error",
JOptionPane.ERROR_MESSAGE);
}
}

public void editNationality()
{
    try {
        // Read existing student data from the file
        BufferedReader reader = new BufferedReader(new FileReader("Studentdata.txt"));
        ArrayList<String> lines = new ArrayList<>();

```

```

String line;

while ((line = reader.readLine()) != null) {

    lines.add(line);

}

reader.close();


// Find the line containing the name and update it
for (int i = 0; i < lines.size(); i++) {

    if (lines.get(i).startsWith("Nationality:")) {

        // Prompt the user for the new name

        String newNationality = JOptionPane.showInputDialog("Enter the new Nationality:");

        if (newNationality == null) {

            return;

        }

        lines.set(i, "Nationality: " + newNationality);

        break; // Once we found and updated the name, we can stop searching

    }

}


// Write the modified data back to the file

BufferedWriter writer = new BufferedWriter(new FileWriter("Studentdata.txt"));

for (String updatedLine : lines) {

    writer.write(updatedLine);

    writer.newLine();

}

writer.close();


JOptionPane.showMessageDialog(null, "Nationality updated successfully!");

```

```

    } catch (IOException e) {
        e.printStackTrace();

        JOptionPane.showMessageDialog(null, "Error occurred while updating Nationality.", "Error",
JOptionPane.ERROR_MESSAGE);
    }
}

```

```

public void editPersonalContact()
{
    try {
        // Read existing student data from the file
        BufferedReader reader = new BufferedReader(new FileReader("Studentdata.txt"));
        ArrayList<String> lines = new ArrayList<>();
        String line;
        while ((line = reader.readLine()) != null) {
            lines.add(line);
        }
        reader.close();

        // Find the line containing the name and update it
        for (int i = 0; i < lines.size(); i++) {
            if (lines.get(i).startsWith("Personal Contact Number:")) {
                // Prompt the user for the new name
                String newPersonalContactNumber = JOptionPane.showInputDialog("Enter the new
Personal Contact Number:");

                if (newPersonalContactNumber == null) {
                    return;
                }

                lines.set(i, "Personal Contact Number: " + newPersonalContactNumber);
            }
        }
    }
}

```

```

        break; // Once we found and updated the name, we can stop searching
    }
}

// Write the modified data back to the file
BufferedWriter writer = new BufferedWriter(new FileWriter("Studentdata.txt"));
for (String updatedLine : lines) {
    writer.write(updatedLine);
    writer.newLine();
}
writer.close();

JOptionPane.showMessageDialog(null, "Personal Contact Number updated successfully!");
} catch (IOException e) {
    e.printStackTrace();

    JOptionPane.showMessageDialog(null, "Error occurred while updating Personal Contact
Number.", "Error", JOptionPane.ERROR_MESSAGE);
}
}

public void editEmergencyContact()
{
    try {
        // Read existing student data from the file
        BufferedReader reader = new BufferedReader(new FileReader("Studentdata.txt"));
        ArrayList<String> lines = new ArrayList<>();
        String line;
        while ((line = reader.readLine()) != null) {

```

```

        lines.add(line);
    }
    reader.close();

    // Find the line containing the name and update it
    for (int i = 0; i < lines.size(); i++) {
        if (lines.get(i).startsWith("Emergency Contact Number:")) {
            // Prompt the user for the new name
            String newEmergencyContactNumber = JOptionPane.showInputDialog("Enter the new
Emergency Contact Number:");
            if (newEmergencyContactNumber == null) {
                return;
            }
            lines.set(i, "Emergency Contact Number: " + newEmergencyContactNumber);
            break; // Once we found and updated the name, we can stop searching
        }
    }

    // Write the modified data back to the file
    BufferedWriter writer = new BufferedWriter(new FileWriter("Studentdata.txt"));
    for (String updatedLine : lines) {
        writer.write(updatedLine);
        writer.newLine();
    }
    writer.close();

    JOptionPane.showMessageDialog(null, "Emergency Contact Number updated successfully!");
} catch (IOException e) {

```

```

        e.printStackTrace();

        JOptionPane.showMessageDialog(null, "Error occurred while updating Emergency Contact
Number.", "Error", JOptionPane.ERROR_MESSAGE);

    }
}
}

```

```

class editStudentinfoGUI extends javax.swing.JFrame {

    editStudentinfo editinfo;

    public editStudentinfoGUI() {

        editinfo = new editStudentinfo();

        initComponents();

        Color col = new Color(245,210,245);

        getContentPane().setBackground(col);

    }
}

```

```

@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code"> //GEN-BEGIN: initComponents
private void initComponents() {

    jPanel1 = new javax.swing.JPanel();
    jLabel1 = new javax.swing.JLabel();
    jButton1 = new javax.swing.JButton();
    jButton2 = new javax.swing.JButton();
    jButton3 = new javax.swing.JButton();
    jButton4 = new javax.swing.JButton();
    jButton5 = new javax.swing.JButton();
    jButton6 = new javax.swing.JButton();
}

```

```

jButton7 = new javax.swing.JButton();
jButton8 = new javax.swing.JButton();
NameButton = new javax.swing.JButton();
jButton10 = new javax.swing.JButton();
jButton11 = new javax.swing.JButton();
jButton12 = new javax.swing.JButton();
logoutButton = new javax.swing.JButton();

setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
getContentPane().setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());

jPanel1.setBackground(new java.awt.Color(255, 210, 255));
jPanel1.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());

jLabel1.setFont(new java.awt.Font("Segoe Print", 1, 18)); // NOI18N
jLabel1.setText("Select the field you want to edit");
jPanel1.add(jLabel1, new org.netbeans.lib.awtextra.AbsoluteConstraints(150, 30, -1, -1));

jButton1.setBackground(new java.awt.Color(255, 204, 255));
jButton1.setText("Nationality");
jButton1.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton1ActionPerformed(evt);
    }
});
jPanel1.add(jButton1, new org.netbeans.lib.awtextra.AbsoluteConstraints(230, 90, 161, 59));

jButton2.setBackground(new java.awt.Color(255, 204, 255));

```



```

jButton2.setText("Personal Contact Number");
jButton2.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton2ActionPerformed(evt);
    }
});
jPanel1.add(jButton2, new org.netbeans.lib.awtextra.AbsoluteConstraints(420, 90, 179, 60));

```

```

jButton3.setBackground(new java.awt.Color(255, 204, 255));
jButton3.setText("Address");
jButton3.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton3ActionPerformed(evt);
    }
});
jPanel1.add(jButton3, new org.netbeans.lib.awtextra.AbsoluteConstraints(230, 230, 161, 59));

```

```

jButton4.setBackground(new java.awt.Color(255, 204, 255));
jButton4.setText("Date Of Birth");
jButton4.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton4ActionPerformed(evt);
    }
});
jPanel1.add(jButton4, new org.netbeans.lib.awtextra.AbsoluteConstraints(230, 160, 161, 59));

```

```

jButton5.setBackground(new java.awt.Color(255, 204, 255));
jButton5.setText("ID");

```

```

jButton5.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton5ActionPerformed(evt);
    }
});
jPanel1.add(jButton5, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 160, 160, 60));

```

```

jButton6.setBackground(new java.awt.Color(255, 204, 255));
jButton6.setText("Email");
jButton6.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton6ActionPerformed(evt);
    }
});
jPanel1.add(jButton6, new org.netbeans.lib.awtextra.AbsoluteConstraints(420, 160, 179, 60));

```

```

jButton7.setBackground(new java.awt.Color(255, 204, 255));
jButton7.setText("Emergency Contact Number");
jButton7.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton7ActionPerformed(evt);
    }
});
jPanel1.add(jButton7, new org.netbeans.lib.awtextra.AbsoluteConstraints(420, 230, -1, 60));

```

```

jButton8.setBackground(new java.awt.Color(255, 204, 255));
jButton8.setText("Gender");
jButton8.addActionListener(new java.awt.event.ActionListener() {

```

```

    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton8ActionPerformed(evt);
    }
});

jPanel1.add(jButton8, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 300, 160, 60));

NameButton.setBackground(new java.awt.Color(255, 204, 255));
NameButton.setText("Name");
NameButton.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        NameButtonActionPerformed(evt);
    }
});

jPanel1.add(NameButton, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 90, 160, 59));

jButton10.setBackground(new java.awt.Color(255, 204, 255));
jButton10.setText("Age");
jButton10.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton10ActionPerformed(evt);
    }
});

jPanel1.add(jButton10, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 230, 160, 60));

jButton11.setBackground(new java.awt.Color(255, 204, 255));
jButton11.setIcon(new javax.swing.ImageIcon(getClass().getResource("/images/off.png"))); //
NOI18N
jButton11.addActionListener(new java.awt.event.ActionListener() {

```

```

    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton11ActionPerformed(evt);
    }
});

jPanel1.add(jButton11, new org.netbeans.lib.awtextra.AbsoluteConstraints(510, 0, 84, 80));

jButton12.setBackground(new java.awt.Color(255, 204, 255));
jButton12.setIcon(new javax.swing.ImageIcon(getClass().getResource("/images/back.png"))); // NOI18N
jButton12.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        jButton12ActionPerformed(evt);
    }
});

jPanel1.add(jButton12, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 390, 80, 75));

logoutButton.setBackground(new java.awt.Color(255, 204, 255));
logoutButton.setIcon(new
javax.swing.ImageIcon(getClass().getResource("/images/logout.png"))); // NOI18N
logoutButton.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        logoutButtonActionPerformed(evt);
    }
});

jPanel1.add(logoutButton, new org.netbeans.lib.awtextra.AbsoluteConstraints(510, 390, 77,
80));

getContentPane().add(jPanel1, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 0, 680,
520));

```

```

    pack();

    setLocationRelativeTo(null);

} // </editor-fold> // GEN-END: initComponents

    private void jButton7ActionPerformed(java.awt.event.ActionEvent evt) { // GEN-
FIRST:event_jButton7ActionPerformed

        editinfo.editEmergencyContact();

    } // GEN-LAST:event_jButton7ActionPerformed

    private void NameButtonActionPerformed(java.awt.event.ActionEvent evt) { // GEN-
FIRST:event_NameButtonActionPerformed

        editinfo.editName();

    } // GEN-LAST:event_NameButtonActionPerformed

    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) { // GEN-
FIRST:event_jButton1ActionPerformed

        editinfo.editNationality();

    } // GEN-LAST:event_jButton1ActionPerformed

    private void jButton5ActionPerformed(java.awt.event.ActionEvent evt) { // GEN-
FIRST:event_jButton5ActionPerformed

        editinfo.editID();

    } // GEN-LAST:event_jButton5ActionPerformed

    private void jButton3ActionPerformed(java.awt.event.ActionEvent evt) { // GEN-
FIRST:event_jButton3ActionPerformed

        editinfo.editAddress();

    } // GEN-LAST:event_jButton3ActionPerformed

```

```
private void jButton10ActionPerformed(java.awt.event.ActionEvent evt) {GEN-
FIRST:event_jButton10ActionPerformed
```

```
    editinfo.editAge();
```

```
}//GEN-LAST:event_jButton10ActionPerformed
```

```
private void jButton4ActionPerformed(java.awt.event.ActionEvent evt) {GEN-
FIRST:event_jButton4ActionPerformed
```

```
    editinfo.editdob();
```

```
}//GEN-LAST:event_jButton4ActionPerformed
```

```
private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {GEN-
FIRST:event_jButton2ActionPerformed
```

```
    editinfo.editPersonalContact();
```

```
}//GEN-LAST:event_jButton2ActionPerformed
```

```
private void jButton6ActionPerformed(java.awt.event.ActionEvent evt) {GEN-
FIRST:event_jButton6ActionPerformed
```

```
    editinfo.editEmail();
```

```
}//GEN-LAST:event_jButton6ActionPerformed
```

```
private void jButton12ActionPerformed(java.awt.event.ActionEvent evt) {GEN-
FIRST:event_jButton12ActionPerformed
```

```
    this.toBack();
```

```
}//GEN-LAST:event_jButton12ActionPerformed
```

```
private void jButton11ActionPerformed(java.awt.event.ActionEvent evt) {GEN-
FIRST:event_jButton11ActionPerformed
```

```
    exit(0);
```

```
}//GEN-LAST:event_jButton11ActionPerformed
```

```
private void logoutButtonActionPerformed(java.awt.event.ActionEvent evt) {GEN-
FIRST:event_logoutButtonActionPerformed
```

```
    this.toBack();
```

```

    LoginPage login = new LoginPage();
    login.setVisible(true);
    login.toFront();
} //GEN-LAST:event_logoutButtonActionPerformed

// Variables declaration - do not modify//GEN-BEGIN:variables
private javax.swing.JButton NameButton;
private javax.swing.JButton jButton1;
private javax.swing.JButton jButton10;
private javax.swing.JButton jButton11;
private javax.swing.JButton jButton12;
private javax.swing.JButton jButton2;
private javax.swing.JButton jButton3;
private javax.swing.JButton jButton4;
private javax.swing.JButton jButton5;
private javax.swing.JButton jButton6;
private javax.swing.JButton jButton7;
private javax.swing.JButton jButton8;
private javax.swing.JLabel jLabel1;
private javax.swing.JPanel jPanel1;
private javax.swing.JButton logoutButton;
// End of variables declaration//GEN-END:variables
}

```

Design Phase :

Login Page :



Student Home Page :



Teacher Home Page :



CRUD :

Enter Your Particulars

Name

Enter your Name

Nationality

Enter your Nationality

ID

Enter your ID

Address

Enter your Address

Age

Enter your Age

Date Of Birth

Enter your DOB

Gender

Enter your Gender

male

Personal Contact Number

Enter your Contact

Email

Enter your Email

Emergency Contact Number

Enter your Contact

Save

Reset

Select the field you want to edit

Name

Nationality

Personal Contact Numb...

ID

Date Of Birth

Email

Age

Address

Emergency Contact Number

Gender

Save

Cancel

Division of Responsibilities:

- ❖ **Aqsa Ahmed** was responsible for compiling the document.
- ❖ **Simaab Malik** took charge of providing the interfaces and implemented the sprint.
- ❖ **Hoor Spogmay** took charge of providing the code.

Test Cases

Test Case ID	Test Case Description	Test Steps	Pre-Condition	Test Data	Expected Result (post condition)	Status	Assumptions
TC-1	Search for a student record in a database.	1. Navigate to the student database. 2. Enter "Umaira Malik" in the search field. 3. Click on the "Search" button.	The database system should be operational.	Student Name = Umaira Malik	All matching student records should be displayed.	Pass/Fail	The student record exists in the database.
TC-2	View detailed information of a student record.	1. Select the student record "Umaira Malik" from the list. 2. Click on "View Details."	The selected student record must exist in the database.	Selected Student = Umaira Malik	Details (Name, Enrollment number, Attendance, Fee status, Grades) should be displayed.	Pass/Fail	The detailed information for the selected student is complete and accessible.

TC-3	Add a new student record.	1. Go to "Add New Student" form. 2. Enter student details (Name, Enrollment). 3. Click "Save."	The system must allow adding new student records.	Name = Simaab Malik Enrollment = 23235	The new student record should be successfully added to the database.	Pass/Fail	The provided data is valid and follows the required format for database entries.
TC-4	Update a student record.	1. Search for the student with Enrollment = 23235. 2. Select "Edit" option. 3. Update the email field. 4. Click "Save."	The student record should already exist in the database.	Enrollment = 23235 New Email = simbamalik05@gmail.com	The student's email should be updated.	Pass/Fail	The system permits email updates, and the new email format is valid.

TC -5	Remove a student record.	1. Search for the student with Enrollment = 23235. 2. Select "Delete" option. 3. Confirm deletion.	The student record should exist in the database.	Enrollment = 23235	Selected student record should be removed from the database.	Pass/Fail	The system supports deletion of existing student records.
TC -6	Check attendance of a specific student.	1. Navigate to the Attendance module. 2. Search using Enrollment = 23235. 3. View the attendance details.	Attendance records must exist for the student.	Enrollment = 23235	The attendance record for the specific student should be displayed. .	Pass/Fail	Attendance is recorded accurately and accessible for students.

TC-7	Generate fee challan for a student.	1. Go to "Fee Management" module. 2. Search for Enrollment = 23235. 3. Click "Generate Challan."	Student record must exist in the database.	Enrollment = 23235	The fee challan should be generated with accurate calculations.	Pass/Fail	Fee data is accurate, and the system supports fee challan generation.
TC-8	Verify login functionality for a student.	1. Open the login page. 2. Enter valid credentials (Name and Password). 3. Click "Login."	Valid credentials should exist for the student.	Name = Simaab Malik Password = simaab123	The student should successfully log in if the credentials are correct.	Pass/Fail	Login functionality is functional, and credentials are securely stored.

TC-9	Apply late fee charges for a student.	1. Navigate to the "Fee Management" module. 2. Search for Enrollment = 23235. 3. Apply late fee charges.	Fee payment date is marked as "Late."	Enrollment = 23235 Fee Payment Date = Late	Updated fee challan should include late fee charges.	Pass/Fail	The system supports dynamic updates to fee challans and can handle late fee calculations
TC-10	Assign grades for a course.	1. Go to the "Grades" module. 2. Search for Enrollment = 23235. 3. Assign grade for Course = DSA-123. 4. Save.	Student and course records must exist in the database.	Course = DSA-123 Enrollment = 23235 Grade = A	The student should successfully log in if the credentials are correct.	Pass/Fail	The grading system allows for specific grade assignments linked to students and courses.
TC-11	Verify admin access to delete records.	1. Log in with Admin credentials. 2. Search for Student-ID: 23235. 3. Click "Delete" and confirm action.	Admin account should exist and have appropriate permissions.	Admin Login = Admin123 Record = Student-ID: 23235	Admin should be able to delete the specified student record.	Pass/Fail	Admin accounts have sufficient permissions to delete sensitive records securely.

Conclusion:

The development of the Campus Hub system significantly enhances the efficiency and accuracy of managing student data and administrative tasks for educational institutions. By centralizing data and automating processes such as attendance tracking and fee management, the system minimizes manual work and reduces errors. Its user-friendly design ensures easy access for authorized personnel, enabling improved communication and decision-making. This fosters a more organized and secure environment, streamlining educational processes and allowing educators and administrators to focus more on core activities.

The software is built using Agile process model. The framework of Agile “Scrum” made it easy to sequentially execute the implementation of the software. The Architecture of this software is Client-Server Architecture.